

Лабораторная работа №25. Современный JavaScript

Цель работы:

- Научиться работать с асинхронным кодом (промисы, `async/await`);
- Понять основы работы с REST API;

Подготовка проекта

1. Откройте **Visual Studio Code**
2. Откройте **предыдущий** проект.
3. Создайте его копию, с новым именем:

Lab25_ModernJS

4. Откройте созданную папку в **Visual Studio Code**

Инициализация Git-репозитория

1. Удалите скрытую папку **.git** из репозитория.
2. Инициализируйте Git-репозиторий:

git init

3. Добавьте файлы в индекс:

git add .

4. Создайте первый коммит:

git commit -m "Initial commit"

Шаг 10. Async/Await

10.1. Теоретическая часть

async/await — современный способ работы с асинхронным кодом.

Преимущества:

- **Читаемость** — код выглядит как синхронный
- **Обработка ошибок** — привычный `try/catch`
- **Debugging** — проще отлаживать

Правила:

- **async** перед функцией означает, что она возвращает промис
- **await** можно использовать только внутри **async** функций
- **await** приостанавливает выполнение до получения результата

10.2. Базовое использование async/await

В `main.js` добавьте:

Добавим вывод заголовка раздела:

```
console.log("Async/Await");
```

Простая async функция:

Создадим асинхронную функцию `greet`. Ключевое слово `async` автоматически заставляет функцию возвращать промис:

```
async function greet() {  
  return "Привет!";  
}
```

Вызовем функцию и обработаем результат через `.then()`:

```
greet().then((message) => console.log(message));
```

Запустите программу через терминал:

```
➤$ node js/main.js  
Async/Await  
Привет!
```

Использование await:

Создадим функцию `getWeather`, которая возвращает промис с задержкой (имитация запроса к API погоды):

```
function getWeather() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve({ temp: 22, condition: "Солнечно" });  
    }, 1000);  
  });  
}
```

Создадим `async` функцию `showWeather`, где `await` приостанавливает выполнение до получения результата:

```
async function showWeather() {  
  console.log("Загрузка погоды...");  
  const weather = await getWeather();  
  console.log(`Температура: ${weather.temp}°C, ${weather.condition}`);  
}
```

Вызовем функцию:

```
showWeather();
```

Запустите программу через терминал:

```
➤$ node js/main.js
Async/Await
Загрузка погоды...
Привет!
Температура: 22°C, Солнечно
```

Закомментируйте выше написанный код в main.js.

Обработка ошибок с try/catch:

Создадим функцию **fetchData**, которая может завершиться с ошибкой в зависимости от параметра **shouldFail**:

```
async function fetchData(shouldFail) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldFail) {
        reject("Ошибка при загрузке данных");
      } else {
        resolve({ data: "Важные данные" });
      }
    }, 800);
  });
}
```

Создадим **async** функцию **getData** с обработкой ошибок через **try/catch**:

```
async function getData() {
  try {
    const result = await fetchData(false);
    console.log("Успешно:", result.data);

    const failedResult = await fetchData(true);
    console.log("Это не выполнится");
  } catch (error) {
    console.log("Поймана ошибка:", error);
  }
}
```

Вызовем функцию:

```
getData();
```

Запустите программу через терминал:

```
└─>$ node js/main.js
Успешно: Важные данные
Поймана ошибка: Ошибка при загрузке данных
```

Закомментируйте выше написанный код в main.js.

Последовательное выполнение:

Создадим `async` функцию **cookDinner**, где каждый шаг ждёт завершения предыдущего (готовим блюда по очереди):

```
async function cookDinner() {
  console.log("Начинаем готовить...");

  const pasta = await delay(1000).then(() => "Паста готова");
  console.log(pasta);

  const sauce = await delay(500).then(() => "Соус готов");
  console.log(sauce);

  const salad = await delay(700).then(() => "Салат готов");
  console.log(salad);

  return "Ужин готов!";
}
```

Раскомментируйте функцию `delay()`:

```
// function delay(ms) {
//   return new Promise((resolve) => {
//     setTimeout(() => {
//       resolve(`Прошло ${ms} миллисекунд`);
//     }, ms);
//   });
// }
```

Вызовем функцию:

```
cookDinner().then((result) => console.log(result));
```

Запустите программу через терминал:

```
└─>$ node js/main.js
Начинаем готовить...
Паста готова
Соус готов
Салат готов
Ужин готов!
```

Параллельное выполнение с Promise.all:

Создадим функцию **cookDinnerFast**, где все блюда готовятся одновременно.

Promise.all ждёт завершения всех промисов:

```
async function cookDinnerFast() {  
  console.log("Готовим всё одновременно...");  
  
  const [pasta, sauce, salad] = await Promise.all([  
    delay(1000).then(() => "Паста готова"),  
    delay(500).then(() => "Соус готов"),  
    delay(700).then(() => "Салат готов"),  
  ]);  
  
  console.log(pasta, sauce, salad);  
  return "Ужин готов быстрее!";  
}
```

Вызовем функцию:

```
cookDinnerFast().then((result) => console.log(result));
```

Запустите программу через терминал:

```
➤$ node js/main.js  
Начинаем готовить...  
Готовим всё одновременно...  
Паста готова  
Паста готова Соус готов Салат готов  
Ужин готов быстрее!  
Соус готов  
Салат готов  
Ужин готов!
```

Закомментируйте выше написанный код в main.js.

10.3. Практическое задание

Задание:

1. Создайте **async** функцию **processOrder**
2. Она должна последовательно выполнить: проверку наличия товара, расчёт стоимости, подтверждение заказа
3. Каждый шаг возвращает промис с задержкой
4. Используйте try/catch для обработки ошибок

Практическое задание

1. Сделайте скриншот/ы файла **main.js** и сохраните в папку **img** с именем:
step10_asyncAwaitLab25_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "async/await in JS"

git push

Шаг 11. Fetch API - Работа с REST API

11.1. Теоретическая часть

Fetch API — современный способ отправки HTTP-запросов.

REST API — архитектурный стиль для создания веб-сервисов.

Типичные операции:

- **GET** — получить данные
- **POST** — создать новые данные
- **PUT** — обновить данные
- **DELETE** — удалить данные

11.2. Работа с JSONPlaceholder

Мы будем использовать бесплатный тестовый API:

<https://jsonplaceholder.typicode.com>

В **main.js** добавьте:

Добавим вывод заголовка раздела:

```
console.log("Fetch API");
```

Простой GET-запрос:

Создадим **async** функцию **getUsers** для получения списка пользователей с сервера:

```
async function getUsers() {
  try {
    // Отправляем GET-запрос к API
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    // Проверяем успешность запроса
    if (!response.ok) {
      throw new Error(`HTTP ошибка! Статус: ${response.status}`);
    }
    // Преобразуем ответ в JSON
    const users = await response.json();
    // Выводим первых 3 пользователей
    console.log("Первые 3 пользователя:");
    users.slice(0, 3).forEach((user) => {
      console.log(`- ${user.name} (${user.email})`);
    });
  } catch (error) {
    console.log("Ошибка при загрузке пользователей:", error.message);
  }
}
```

Вызовем функцию:

```
getUsers();
```

Запустите программу через терминал:

```
➤$ node js/main.js
Fetch API
Первые 3 пользователя:
- Leanne Graham (Sincere@april.biz)
- Ervin Howell (Shanna@melissa.tv)
- Clementine Bauch (Nathan@yesenia.net)
```

Закомментируйте выше написанный код в main.js.

Получение одного пользователя:

Создадим функцию `getUserById`, которая получает данные конкретного пользователя по его ID:

```
async function getUserById(id) {
  try {
    // Используем шаблонную строку для подстановки ID в URL
    const response = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
    const user = await response.json();
    // Выводим информацию о пользователе
    console.log(`Пользователь #${id}:`);
    console.log(`Имя: ${user.name}`);
    console.log(`Город: ${user.address.city}`);
    console.log(`Компания: ${user.company.name}`);
  } catch (error) {
    console.log("Ошибка:", error.message);
  }
}
```

Вызовем функцию для пользователя с ID = 1:

```
getUserById(1);
```

Запустите программу через терминал:

```
➤$ node js/main.js
Пользователь #1:
Имя: Leanne Graham
Город: Gwenborough
Компания: Romaguera-Crona
```

Закомментируйте выше написанный код в main.js.

POST-запрос (создание данных):

Создадим функцию `createPost` для отправки новой записи на сервер:

```

async function createPost(){
  try {
    // Создаём объект с данными новой записи
    const newPost = {
      title: "Моя первая запись",
      body: "Это содержание моей первой записи в блоге",
      userId: 1,
    };
    // Отправляем POST-запрос с настройками
    const response = await fetch("https://jsonplaceholder.typicode.com/posts", {
      method: "POST", // Метод HTTP-запроса
      headers: {
        "Content-Type": "application/json", // Указываем тип данных
      },
      body: JSON.stringify(newPost), // Преобразуем объект в JSON-строку
    });
    // Получаем ответ сервера
    const createdPost = await response.json();
    console.log("Создана новая запись:");
    console.log("ID:", createdPost.id);
    console.log("Заголовок:", createdPost.title);
  } catch (error) {
    console.log("Ошибка при создании записи:", error.message);
  }
}

```

Вызовем функцию:

```
createPost();
```

Запустите программу через терминал:

```

└─>$ node js/main.js
Создана новая запись:
ID: 101
Заголовок: Моя первая запись

```

Закомментируйте выше написанный код в main.js.

Практическое задание

1. Сделайте скриншот/ы файла **main.js** и сохраните в папку **img** с именем:

step11_fetchApiLab25_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "fetch API implementation"

git push

Шаг 12. Optional Chaining и Nullish Coalescing

12.1. Теоретическая часть

Optional Chaining (?.) — безопасный доступ к вложенным свойствам объектов.

Nullish Coalescing (??) — выбор значения по умолчанию.

Эти операторы появились в ES2020 и решают частые проблемы с **null** и **undefined**.

Сравнение с C#:

C#	JavaScript
obj?.Property	obj?.property
value ?? defaultValue	value ?? defaultValue

12.2. Optional Chaining (?.)

В **main.js** добавьте:

Добавим вывод заголовка раздела:

```
console.log("Optional Chaining");
```

Проблема без optional chaining:

Создадим два объекта пользователей: один с полным адресом, другой — без:

```
const user1 = {
  name: "Андрей",
  address: {
    city: "Волжский",
    street: "Пушкина",
  },
};
```

```
const user2 = {
  name: "Дмитрий",
  // address отсутствует
};
```

Попытка обратиться к вложенному свойству у **user2**:

```
// const city = user2.address.city;
```

Вызовет ошибку:

```
└─>$ node js/main.js
Optional Chaining
/home/denis/Project/ISRPO_Fullstack/Lab17_ModernJS_Leontev/js/main.js:401
const city = user2.address.city;
                        ^
TypeError: Cannot read properties of undefined (reading 'city')
```

Старый способ решения — проверка каждого уровня вложенности:

```
const city1 = user2.address && user2.address.city;
console.log("Город (старый способ):", city1); // undefined
```

Запустите программу через терминал:

```
└─>$ node js/main.js
Optional Chaining
Город (старый способ): undefined
```

Новый способ с optional chaining:

Используем оператор `?.` для безопасного доступа. Если `address` равен `null` или `undefined`, вернётся `undefined` без ошибки:

```
const city2 = user2.address?.city;
console.log("Город (новый способ):", city2); // undefined

const street = user1.address?.street;
console.log("Улица:", street); // 'Пушкина'
```

Запустите программу через терминал:

```
└─>$ node js/main.js
Optional Chaining
Город (старый способ): undefined
Город (новый способ): undefined
Улица: Пушкина
```

Закомментируйте выше написанный код в main.js.

Optional chaining с методами:

Создадим объект администратора с методом и объект гостя без метода:

```
const admin = {
  name: "Администратор",
  permissions: {
    canDelete: () => true,
  },
};
```

```
const guest = {  
  name: "Гость",  
};
```

Используем `?.()` для безопасного вызова метода:

```
console.log("Админ может удалять?", admin.permissions?.  
canDelete?.()); // true  
console.log("Гость может удалять?", guest.permissions?.  
canDelete?.()); // undefined
```

Запустите программу через *терминал*:

```
└─>$ node js/main.js  
Админ может удалять? true  
Гость может удалять? undefined
```

Закомментируйте выше написанный код в `main.js`.

Optional chaining с массивами:

Создадим объект компании с массивом сотрудников и объект стартапа без сотрудников:

```
const company = {  
  name: "Tech Corp",  
  employees: [  
    { name: "Надежда", role: "Developer" },  
    { name: "Анна", role: "Designer" },  
  ],  
};
```

```
const startup = {  
  name: "New Startup",  
};
```

Используем `?.[индекс]` для безопасного доступа к элементу массива:

```
console.log("Первый сотрудник:", company.employees?.  
[0]?.name);  
console.log("Первый сотрудник стартапа:", startup.  
employees?.[0]?.name);
```

Запустите программу через *терминал*:

```
└─>$ node js/main.js
Первый сотрудник: Надежда
Первый сотрудник стартапа: undefined
```

Закомментируйте выше написанный код в `main.js`.

12.3. Nullish Coalescing (??)

В `main.js` добавьте:

Добавим вывод заголовка раздела:

```
console.log("Nullish Coalescing");
```

Разница между `||` и `??`:

Создадим переменные с разными falsy значениями (0, пустая строка, false, null, undefined):

```
const value1 = 0;
const value2 = "";
const value3 = false;
const value4 = null;
const value5 = undefined;
```

Сравним поведение операторов `||` и `??`:

Оператор `||` возвращает правое значение для любого falsy (0, "", false, null, undefined):

```
console.log('value1 || "default":', value1 || "default");
console.log('value2 || "default":', value2 || "default");
console.log('value3 || "default":', value3 || "default");
```

- 'default' (0 считается falsy)
- 'default' (" считается falsy)
- 'default' (false считается falsy)

```
└─>$ node js/main.js
Nullish Coalescing
value1 || "default": default
value2 || "default": default
value3 || "default": default
```

Оператор `??` возвращает правое значение ТОЛЬКО для **null** и **undefined**:

```
console.log('value1 ?? "default":', value1 ?? "default");
console.log('value2 ?? "default":', value2 ?? "default");
console.log('value3 ?? "default":', value3 ?? "default");
console.log('value4 ?? "default":', value4 ?? "default");
console.log('value5 ?? "default":', value5 ?? "default");
```

- 0 (это валидное значение)
- " (это валидное значение)
- false (это валидное значение)
- 'default' (null заменяется)
- 'default' (undefined заменяется)

```
➤ $ node js/main.js
Nullish Coalescing
value1 ?? "default": 0
value2 ?? "default":
value3 ?? "default": false
value4 ?? "default": default
value5 ?? "default": default
```

Закомментируйте выше написанный код в main.js.

Практический пример:

Создадим функцию **displayUserSettings**, которая принимает объект настроек и задаёт значения по умолчанию с помощью **??**. Используем optional chaining для доступа к свойствам и nullish coalescing для значений по умолчанию

```
function displayUserSettings(settings) {
  const theme = settings?.theme ?? "light";
  const fontSize = settings?.fontSize ?? 14;
  const notifications = settings?.notifications ?? true;

  console.log("Настройки пользователя:");
  console.log("Тема:", theme);
  console.log("Размер шрифта:", fontSize);
  console.log("Уведомления:", notifications);
}
```

Вызовем функцию с разными объектами настроек:

```
// Используются указанные значения
displayUserSettings({ theme: "dark", fontSize: 16 });
// notifications = false (не заменяется на true!)
displayUserSettings({ notifications: false });
// Все значения по умолчанию
displayUserSettings({});
```

Закомментируйте выше написанный код в main.js.

Комбинирование операторов:

Создадим объект с глубокой вложенностью (имитация ответа API):

```
const apiResponse = {
  data: {
    user: {
      profile: {
        settings: {
          language: "ru",
        },
      },
    },
  },
};
```

Используем ?. для безопасного доступа и ?? для значения по умолчанию:

```
const language = apiResponse?.data?.user?.profile?.
settings?.language ?? "en";
console.log("Язык:", language); // 'ru'
```

Запустите программу через *терминал*:

```
➤$ node js/main.js
Язык: ru
```

Проверим с пустым объектом:

```
const emptyResponse = {};
const defaultLanguage = emptyResponse?.data?.user?.profile?.
settings?.language ?? "en";
console.log("Язык по умолчанию:", defaultLanguage); // 'en'
```

Запустите программу через *терминал*:

```
➤$ node js/main.js
Язык: ru
Язык по умолчанию: en
```

Закомментируйте выше написанный код в main.js.

12.4. Практическое задание

Задание:

1. Создайте объект **order** с вложенными свойствами (customer, shipping, payment)

2. Используйте optional chaining для безопасного доступа к свойствам
3. Используйте nullish coalescing для значений по умолчанию
4. Создайте функцию **displayOrder**, которая красиво выводит информацию о заказе

Практическое задание

1. Сделайте скриншот файла **main.js** и сохраните в папку **img** с именем: **step12_optionalLab25_FIO.png**
2. Выполните следующие команды:

git add .

git commit -m "optional chaining and nullish coalescing"

git push

Самостоятельное задание. Конвертер валют

Реализуйте самостоятельно следующую программу - Конвертер валют.

Функционал:

1. Input для суммы
2. Выпадающий список валют
3. Кнопка «Конвертировать»
4. Вывод результата на страницу
5. Минимальная проверка данных (например, пустой ввод)

Пример коммитов:

1. *Initial project: currency converter structure*
2. *Added CSS styling*
3. *Implemented currency conversion logic*
4. *Added input validation and result display*

Скриншот: *currency_FIO.png*